



SAPIENZA
UNIVERSITÀ DI ROMA

DIPARTIMENTO DI INGEGNERIA INFORMATICA, AUTOMATICA E
GESTIONALE

Autonomous Library Reshelving: A Project Proposal

PLANNING AND REASONING

Professor:

Andrea Marrella

Students:

Vano Mazashvili

Matricola 1993251

Academic Year 2025/2026

Group Composition: Individual project: Vano Mazashvili (1993251) (part-time)

1 Domain

The project models the logistics of an **autonomous library reshelving service**. A small fleet of robots make sure to collect and distribute the returned books from the central return cart in the library to their respective shelves.

The library is represented as a graph of zones connected by the paths. Shelf zones hold books from a single category (sci-fi, history, etc.). Each book is attributed by the precisely one category. Every book must be placed in the zone respective to their category.

The robots are battery powered and consume energy while navigating the library. Re-shelving will require a robot to recharge at one of the charging docks placed at fixed zones.

We have following actions:

- *navigate*: between adjacent zones
- *pick up*: a book from a given zone
- *shelve*: a book on its respective shelf
- *recharge*: at dock

Objective is to correctly shelf every returned book, with the minimum total cost. category-matching constraint and the battery cycle together force the planner to interleave navigation, deliveries, and recharging trips, generating non-trivial planning structure within plain STRIPS-style PDDL.

Predicates and Fluents

- $\text{RobotAt}(r, z), \text{BookAt}(b, z), \text{Carrying}(r, b), \text{Shelved}(b), \text{BatteryLow}(r)$ – *fluents*.
- $\text{Path}(z_1, z_2), \text{StnAt}(z), \text{BookCategory}(b, c), \text{ShelfCategory}(z, c)$ – *rigid*.

Actions

- $\text{navigate}(r, z_1, z_2)$. *Pre*: $\text{RobotAt}(r, z_1) \wedge \text{Path}(z_1, z_2) \wedge \neg \text{BatteryLow}(r)$.
Eff: $\text{RobotAt}(r, z_2) \wedge \neg \text{RobotAt}(r, z_1)$.
- $\text{pickup}(r, b, z)$. *Pre*: $\text{RobotAt}(r, z) \wedge \text{BookAt}(b, z) \wedge \neg \exists b'. \text{Carrying}(r, b') \wedge \neg \text{Shelved}(b)$. *Eff*: $\text{Carrying}(r, b) \wedge \neg \text{BookAt}(b, z)$.

- `shelve(r, b, z)`. *Pre*: $\text{RobotAt}(r, z) \wedge \text{Carrying}(r, b) \wedge \text{ShelfCategory}(z, c) \wedge \text{BookCategory}(b, c) \wedge \neg \text{BatteryLow}(r)$. *Eff*: $\text{Shelved}(b) \wedge \text{BookAt}(b, z) \wedge \neg \text{Carrying}(r, b) \wedge \text{BatteryLow}(r)$.
- `recharge(r, z)`. *Pre*: $\text{RobotAt}(r, z) \wedge \text{StnAt}(z) \wedge \text{BatteryLow}(r)$. *Eff*: $\neg \text{BatteryLow}(r)$.

2 Problem Instances

The three instances scale both the geometric difficulty (more zones, longer paths) and the type-matching difficulty (more categories, more careful routing decisions). Problem 3 is sized conservatively to remain solvable with at least one heuristic configuration within reasonable time bounds.

Instance	Robots	Zones	Books	Categ.	Docks	Notable structure
Problem 1 (easy)	1	5	2	2	1	Linear topology, single agent, minimal recharging
Problem 2 (medium)	2	8	4	3	1	Branching topology, multi-agent, at least one mid-task recharge
Problem 3 (hard)	2	12	6	4	2	Grid topology, multiple recharges interleaved with deliveries

3 Planners

Fast-Downward¹ as the primary planner, supporting all course heuristics. **planning.domains**² as a secondary cross-check for reproducibility.

4 Search Heuristics

A comparative analysis will be performed across the following heuristics:

- **Blind** – baseline (admissible).
- $h^{\max}$ – admissible delete-relaxation heuristic.
- h^{add} – typically inadmissible, often more informative.
- h^{FF} – relaxed-plan extraction; typically the best practical choice for satisficing search.

¹<https://www.fast-downward.org/>

²<http://planning.domains/>

For each problem the comparison will report search time, expanded nodes, plan length, and plan cost, connecting empirical results to the theoretical ordering $h^{\max} \leq h^+ \leq h^{\text{FF}} \leq h^{\text{add}}$ presented in lectures. The PDDL specification will remain STRIPS-style to ensure compatibility with all the heuristics above.

5 IndiGolog Reasoning Tasks

The same domain will be re-formalised as a Basic Action Theory in the Situation Calculus and implemented in IndiGolog using the SWI-Prolog interpreter from the official repository³, with explicit successor-state axioms for each fluent and rigid predicates encoded as Prolog facts.

Task 1: Legality of an Action Sequence

Verify whether a proposed sequence of robot actions is executable in the initial situation, by checking that each action's preconditions hold in the situation reached by its prefix. For example, given:

```
pickup(r1, b1, return_cart); navigate(r1, return_cart, shelf_fiction);  
shelve(r1, b1, shelf_fiction); recharge(r1, dock_1)
```

the system will verify that each step is legal. Both legal and illegal example sequences will be presented.

Task 2: Projection of a Plan

Given an initial state and a sequence of actions, project the resulting situation and verify whether a chosen condition holds in it. Used to verify properties of candidate plans, e.g., “after this sequence, all books are shelved and at least one robot has full battery”. Both holding and non-holding cases will be presented.

Task 3: Reactive Controller for Unexpected Events

A controller, written using `prioritized_interrupts` and conditional planning, that responds to exogenous events arriving during execution. The controller will handle:

- **Urgent request:** a researcher requests immediate retrieval of a specific book, preempting routine reshelving.
- **New return:** a new book is added to the return cart mid-execution and must be incorporated into the plan.

³<https://github.com/ai-krml-uoft/indigolog>

- **Robot failure:** a robot becomes unavailable and its remaining tasks must be redistributed.

The IndiGolog implementation will demonstrate the reactive controller resuming routine work after handling each event, keeping the library in a consistent state.